Lab Assignment 5: Lexical Analyzer with Symbol Table and Error Reporting

Course: CS3104 - Compiler Design Laboratory
Date: 17/09/2026

Instructions

- Write your name and roll number on top of your source code as comments.
- Source file name must be "Lab7.l".
- Your program must compile cleanly with: flex Lab7.l && gcc lex.yy.c -o lab7 -lfl
- Submit only the source file (Lab7) through the same submission portal used for previous assignments - http://10.22.11.15/~abyaym/2026/CS3104_26/submission/

Problem Statement

A production scanner does more than just print a category for each lexeme: it keeps track of where in the source file it currently is, it remembers every distinct identifier it has seen so far (a symbol table), and when it meets text that cannot be classified as any valid token, it reports a precise, recoverable error instead of crashing or silently swallowing the bad input. In this assignment you will extend a basic Flex token classifier with line-number tracking, a simple in-memory symbol table for identifiers, and lexical error recovery.

Your program must read a C-like source snippet from standard input (or a file) and, in a single pass, do the following: (1) classify every lexeme into KEYWORD, IDENTIFIER, NUMBER, STRING, OPERATOR, or PUNCTUATION and print it together with the line number it occurred on; (2) maintain a symbol table that records, for every distinct identifier, the line on which it first appeared and how many times it has appeared in total; and (3) when a character or lexeme cannot be classified as any valid token — an illegal character or a malformed numeric literal — report a lexical error with the line number and continue scanning the rest of the file rather than stopping.

Example

Input:

```
int total = 0;
int count = 5;
while (count > 0) {
  total = total + count;
  count = count - 1;
}
char *label = "final total";
float bad = 3.14.15;
int x = 12 @ 3;
```

Output (excerpt):

```
1: int -> KEYWORD
1: total -> IDENTIFIER
1: = -> OPERATOR
1: 0 -> NUMBER
1: ; -> PUNCTUATION
2: int -> KEYWORD
2: count -> IDENTIFIER
...
4: total -> IDENTIFIER
4: = -> OPERATOR
4: total -> IDENTIFIER
```

```
4: + -> OPERATOR
4: count -> IDENTIFIER
...
7: "final total" -> STRING
Lexical Error at line 8: malformed number '3.14.15'
Lexical Error at line 9: unexpected character '@'
9: 3 -> NUMBER
9: ; -> PUNCTUATION

Symbol Table:
  total   first line 1   occurrences 3
  count   first line 2   occurrences 4
  label   first line 7   occurrences 1
  bad     first line 8   occurrences 1
  x       first line 9   occurrences 1
```

Notice that the malformed number and the illegal character each produce a single line-numbered error message, and scanning simply resumes afterwards — the program never stops early, and every identifier's line-1 occurrence and later reuses are all reflected correctly in the final symbol table.

Core Requirements

- **Line tracking:** maintain a running line counter (via %option yylineno or a manual global incremented on every '\n' rule) and prefix every printed token with the line number it was matched on, in the form <line>: <lexeme> -> <CATEGORY>.
- **Keyword recognition:** treat int, float, char, double, if, else, while, for, return, void as KEYWORD.
- **Identifier recognition and symbol table:** [a-zA-Z_][a-zA-Z0-9_]* not matching a keyword is an IDENTIFIER; on every match, look the lexeme up in an in-memory symbol table (a simple array or linked list of {name, first_line, count} is sufficient) — insert a new entry with the current line as first_line if the name is not yet present, otherwise increment its count.
- **Number recognition:** well-formed integers and simple decimals (e.g. 42, 3.14) are NUMBER; a numeric-looking lexeme that is not well-formed (e.g. 3.14.15, 12abc) must be reported as a lexical error rather than accepted or silently split.
- **String literal recognition:** a double-quoted run of characters on a single line (e.g. "final total") is a STRING; a string that is never closed before the line ends is a lexical error, not a token.
- **Operator recognition:** +, -, *, /, =, <, >, plus the two-character operators ==, <=, >=, != as OPERATOR — list multi-character operator patterns before their single-character prefixes so the longest match is chosen correctly.
- **Punctuation recognition:** ; , ( ) { } as PUNCTUATION.
- **Lexical error recovery:** any character that cannot begin a valid token (e.g. @, #, $) must produce "Lexical Error at line <n>: unexpected character '<c>'" and then be skipped so scanning continues with the very next character, instead of the program aborting.
- **End-of-file summary:** once input is exhausted, print the full symbol table in the order identifiers were first seen, showing each name, its first line, and its total occurrence count.

Implementation Guidelines

- Enable %option yylineno in the definitions section (or declare and increment your own global line counter in a \n rule) before writing any rule that prints a line number.
- Keep the symbol table as a fixed-size array of a small struct (name, first_line, count); a linear search on each IDENTIFIER match is simple and fast enough for the input sizes used in this lab.

- Order your keyword alternatives before the generic identifier pattern, and order two-character operator patterns (==, <=, >=, !=) before their single-character counterparts (=, <, >) — otherwise Flex's "first rule wins on a tie" behaviour will misclassify them.
- For malformed numbers, match the well-formed NUMBER pattern first, then add a second, broader pattern (e.g. one that also allows stray dots or trailing letters) placed immediately after it purely to catch and report the malformed case — do not let a malformed lexeme fall through to the generic-character error rule, since that would split it into several confusing single-character errors instead of one clear message.
- For strings, match \"[^\"\n]*\" for the well-formed case and a separate \"[^\"\n]* rule (no closing quote before a newline or EOF) to report the unterminated-string error.
- Write the catch-all illegal-character rule (a single "." pattern) last, after every other rule, so it only fires when nothing more specific has matched.
- Print the symbol table from a rule attached to <<EOF>>, so it is emitted exactly once after the very last token has been processed.

Compilation & Testing

```
$ flex Lab7.l
$ gcc lex.yy.c -o lab7 -lfl
$ ./lab7 < input.c
```

Deliverables

- Lab7.l